

Лабораторная работа №3

Конечный марковский процесс принятия решений

Марковский процесс принятия решений (МППР, англ. Markov decision process, MDP) — математический формализм для марковского дискретного стохастического процесса управления, основа для моделирования последовательного принятия решений в ситуациях, где результаты частично случайны и частично зависят от лица, принимающего решения. МППР используется во множестве областей, включая робототехнику, автоматизированное управление, экономику и производство. Подход обучения с подкреплениями, основанный на данной модели, применяется, например, в нейронной сети AlphaZero.

Марковские процессы принятия решений представляют собой инструмент для постановки задачи обучения, где достижение цели осуществляется через взаимодействие и последовательное принятие решений. Окружающая среда (или просто среда), представляет собой сторону, с которой взаимодействует агент. Агент выбирает действия, в то время как среда реагирует на эти действия и предоставляет новые ситуации для агента. Кроме того, среда генерирует вознаграждения — числовые значения, которые агент стремится максимизировать с течением времени путем выбора действий. Инженерам будут более понятны термины: агент — устройство управления или контроллер, среда — управляемая система, действие — управляющий сигнал.

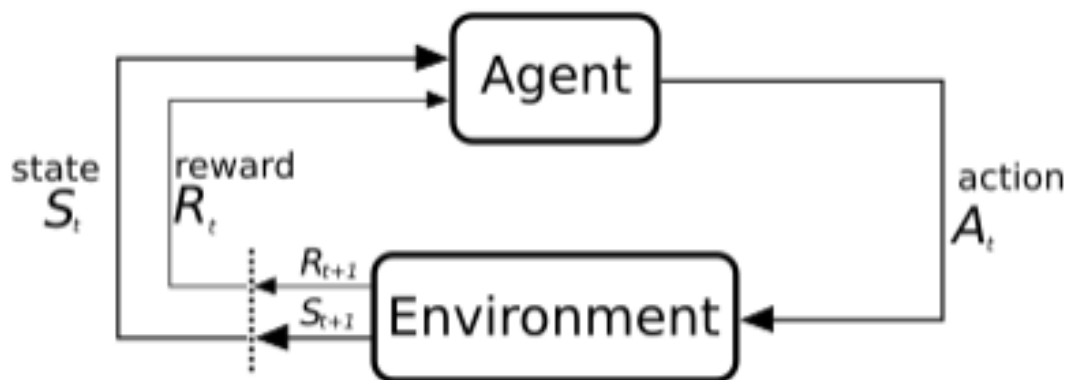


Рисунок 1. Диаграмма обучения с подкреплением в МППР

Марковские процессы принятия решений представляют собой инструмент для постановки задачи обучения, где достижение **цели** осуществляется через взаимодействие и последовательное принятие решений. Окружающая **среда** (или просто среда), представляет собой сторону, с которой взаимодействует **агент**. Агент выбирает действия, в то время как среда реагирует на эти действия и предоставляет новые ситуации для агента. Кроме того, среда генерирует **вознаграждения** — числовые значения, которые агент стремится максимизировать с течением времени путем выбора действий. Инженерам будут более понятны термины: агент — устройство управления или контроллер, среда — управляемая система, действие — управляющий сигнал.

Формально определить марковский процесс принятия решений можно, задав 4-кортеж $(S, A, P(\cdot, \cdot), R(\cdot, \cdot))$, где

- S — конечное множество состояний среды, из которых агент наблюдает $S_t \in S$ в момент времени $t = 0, 1, 2 \dots$,
- $A(s)$ — конечное множество **действий**, доступных из состояния s , из которых агент может выбрать для момента времени t действие $a_t \in A(s)$,
- $P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$ — **вероятность перехода состояний**. То есть, вероятность того, что действие a в состоянии s в момент времени t приведёт в состояние s' в момент $t + 1$,
- $R_a(s, s')$ **вознаграждение**, получаемое после перехода в состояние s' из состояния s при совершении действия a .

Совместно агент и среда порождают траекторию $S_0, A_0, R_0, S_1, A_1, R_1, \dots$

Стратегия π — функция (в общем случае распределение вероятностей), сопоставляющая состоянию действие. При наличии такой функции МППР можно рассматривать как Марковскую цепь.

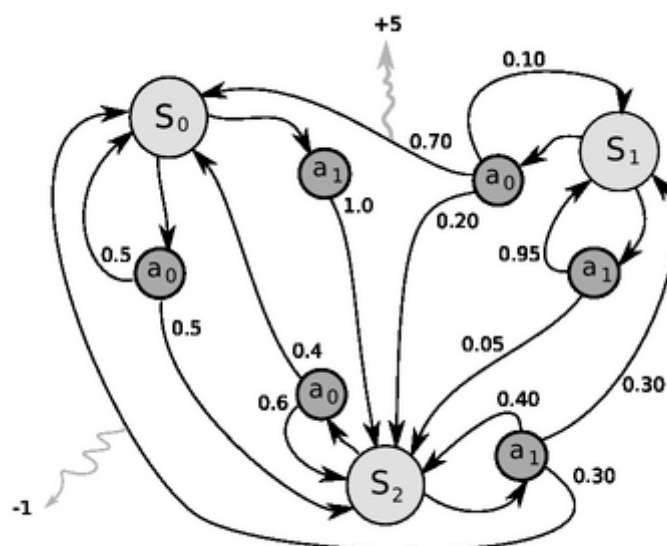


Рисунок 2. Пример МППР с 3 состояниями и 2 действиями

Формализм марковских процессов принятия решений является важной абстракцией задачи обучения целеустремленного агента в процессе взаимодействия. Он позволяет утверждать, что независимо от деталей

механизмов восприятия, памяти и управления, а также от цели, которую преследует агент, любая задача обучения целенаправленному поведению может быть сведена к трем сигналам, которыми агент обменивается с окружающей средой: сигнал, представляющий выбор агента (действие), сигнал причины такого выбора (состояние среды), и сигнал, определяющий цель агента (вознаграждение). Этот формализм не всегда достаточен для описания всех задач обучения принятию решений, но он широко применяется и полезен.

Решить марковский процесс принятия решений означает найти оптимальную стратегию, максимизирующую вознаграждение (**функцию ценности**).

Самая простая функция ценности — это математическое ожидание формального ряда $M \left[\sum_{t=0}^{\infty} R_{a_t}(s_t, s_{t+1}) \right]$, где

$a_t = \pi(s_t)$, а математическое ожидание берётся в соответствии с распределением вероятности $s_{t+1} \sim P_{a_t}(s_t, \cdot)$, но такую функцию можно использовать только если ряд сходится всегда, что обычно означает наличие конечного состояния МППР — такого, что $P_a(s, s) = 1$ и $R_a(s, s) = 0$. Если же сходимость ряда не гарантируется, можно:

- Рассмотреть только конечное число слагаемых $M \left[\sum_{t=0}^N R_{a_t}(s_t, s_{t+1}) \right]$
- Ввести $\gamma \in [0, 1]$ — коэффициент обесценивания (дисконтирования) $M \left[\sum_{t=0}^{\infty} \gamma^t R_{a_t}(s_t, s_{t+1}) \right]$, который контролирует предпочтение агентом мгновенных вознаграждений по сравнению с вознаграждениями в будущем

На практике второй вариант более гибкий, так как учитывает более долгосрочную перспективу и чаще используется именно он.

Для максимизации такого ряда вводят две функции:

- Функция полезности состояния $V_{\pi}(s) = M \left[\sum_{t=0}^{\infty} \gamma^t R_{a_t}(s_t, s_{t+1}) \mid s_0 = s, a_t = \pi(s_t) \right]$, где математическое ожидание берётся в соответствии с распределением $s_{t+1} \sim P_{a_t}(s_t, \cdot)$
- Функция полезности действия $Q_{\pi}(s, a) = M \left[\sum_{t=0}^{\infty} \gamma^t R_{a_t}(s_t, s_{t+1}) \mid s_0 = s, a_0 = a, a_t = \pi(s_t) \forall t \geq 1 \right]$, где математическое ожидание берётся в соответствии с $s_{t+1} \sim P_{a_t}(s_t, \cdot)$

А также их максимумы по всем стратегиям:

- $V_*(s) = \max_{\pi} V_{\pi}(s)$
- $Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$

Можно доказать, что эти функции также являются **функциями полезности состояния** и **полезности действия** соответственно, а также, что они достигаются на детерминированной стратегии. Заметим, что по функции Q_* можно восстановить её стратегию, которая будет оптимальной.

Чтобы дать формальное определение оптимальной стратегии необходимо ввести отношение порядка на множестве стратегий. $\pi_1 \preceq \pi_2 \iff \forall V_{\pi_1}(s) \leq V_{\pi_2}(s), s \in S$. Наибольшая стратегия называется оптимальной.

Можно доказать, что оптимальная стратегия существует.

Алгоритмические реализации

Большинство алгоритмов марковских процессов принятия решений основаны на итерации уравнения Беллмана с фиксированной точкой. Примеры включают итерацию состояния среды (англ. *value iteration*), итерацию стратегии (англ. *policy iteration*), метод временных разностей (англ. *time difference, TD*), Q-обучение и т. д. Анализ этих алгоритмов в табличном случае и случае линейной аппроксимации функции часто использует свойство сжатия оператора Беллмана. В последнее десятилетие нелинейные аппроксимации, такие как нейронные сети, стали более популярными. Однако для нелинейных аппроксимаций функций это свойство сжатия уже не выполняется, что часто приводит к нестабильности. Было предложено множество вариантов и модификаций для стабилизации обучения, например, DQN (англ. *Deep Q-Network* — «глубокая Q-сеть»), A3C (англ. *Asynchronous Advantage Actor-Critic* — «агент-критик с асинхронным преимуществом»). Однако для этих алгоритмов по-прежнему отсутствуют теоретические гарантии.

Задание. Написать код прохождения лабиринта агентом (Python).

Рассмотрим создание алгоритма для решения лабиринта с использованием конечного марковского процесса принятия решений (MDP) и обучения с подкреплением. Ниже представлен пример реализации такого алгоритма на Python с использованием библиотеки `pymru`. В этом примере мы будем использовать метод Q-обучения.

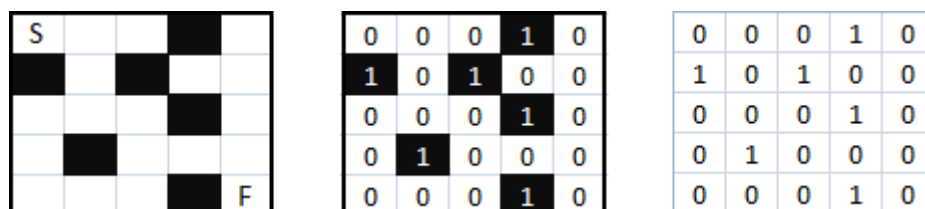


Рисунок 3. Лабиринт и его числовое представление

	0	1	2	3	4
0	0	0	0	1	0
1	1	0	1	0	0
2	0	0	0	1	0
3	0	1	0	0	0
4	0	0	0	1	0

Рисунок 4. Числовое представление лабиринта с координатной сеткой


myCompiler

Enter a title...

 Python



```

1 import numpy as np
2 import random
3 import matplotlib.pyplot as plt
4
5 # Определяем параметры среды
6 class Maze:
7     def __init__(self, maze):
8         self.maze = maze
9         self.n_rows, self.n_cols = maze.shape
10        self.start = (0, 0) # Начальная позиция
11        self.goal = (self.n_rows - 1, self.n_cols - 1) # Целевая позиция
12        self.state = self.start
13
14    def reset(self):
15        self.state = self.start
16        return self.state
17
18    def is_done(self):
19        return self.state == self.goal
20
21    def step(self, action):
22        # Определяем возможные действия
23        actions = [(0, 1), (1, 0), (0, -1), (-1, 0)] # right, down, left, up
24        new_state = (self.state[0] + actions[action][0], self.state[1] + actions[action][1])
25
26        # Проверяем, не вышли ли мы за границы и не в стену
27        if (0 <= new_state[0] < self.n_rows and
28            0 <= new_state[1] < self.n_cols and
29            self.maze[new_state] != 1):
30            self.state = new_state
31

```

```

32     # Награда: +1 за достижение цели, -1 за шаг
33     reward = 1 if self.is_done() else -1
34     return self.state, reward, self.is_done()
35
36 # Q-обучение
37 class QLearning:
38     def __init__(self, maze, alpha=0.1, gamma=0.9, epsilon=0.1):
39         self.maze = maze
40         self.alpha = alpha # Скорость обучения
41         self.gamma = gamma # Дисконтирование будущих наград
42         self.epsilon = epsilon # Вероятность случайного выбора действия
43         self.q_table = np.zeros((maze.n_rows, maze.n_cols, 4)) # Q-таблица
44
45     def choose_action(self, state):
46         if random.uniform(0, 1) < self.epsilon:
47             return random.randint(0, 3) # Случайное действие
48         else:
49             return np.argmax(self.q_table[state]) # Лучшее действие
50
51     def update_q_table(self, state, action, reward, next_state):
52         best_next_action = np.argmax(self.q_table[next_state])
53         td_target = reward + self.gamma * self.q_table[next_state][best_next_action]
54         td_delta = td_target - self.q_table[state][action]
55         self.q_table[state][action] += self.alpha * td_delta
56
57     def train(self, episodes):
58         for episode in range(episodes):
59             state = self.maze.reset()
60             while not self.maze.is_done():
61                 action = self.choose_action(state)
62                 next_state, reward, done = self.maze.step(action)
63                 self.update_q_table(state, action, reward, next_state)
64                 state = next_state

```

```

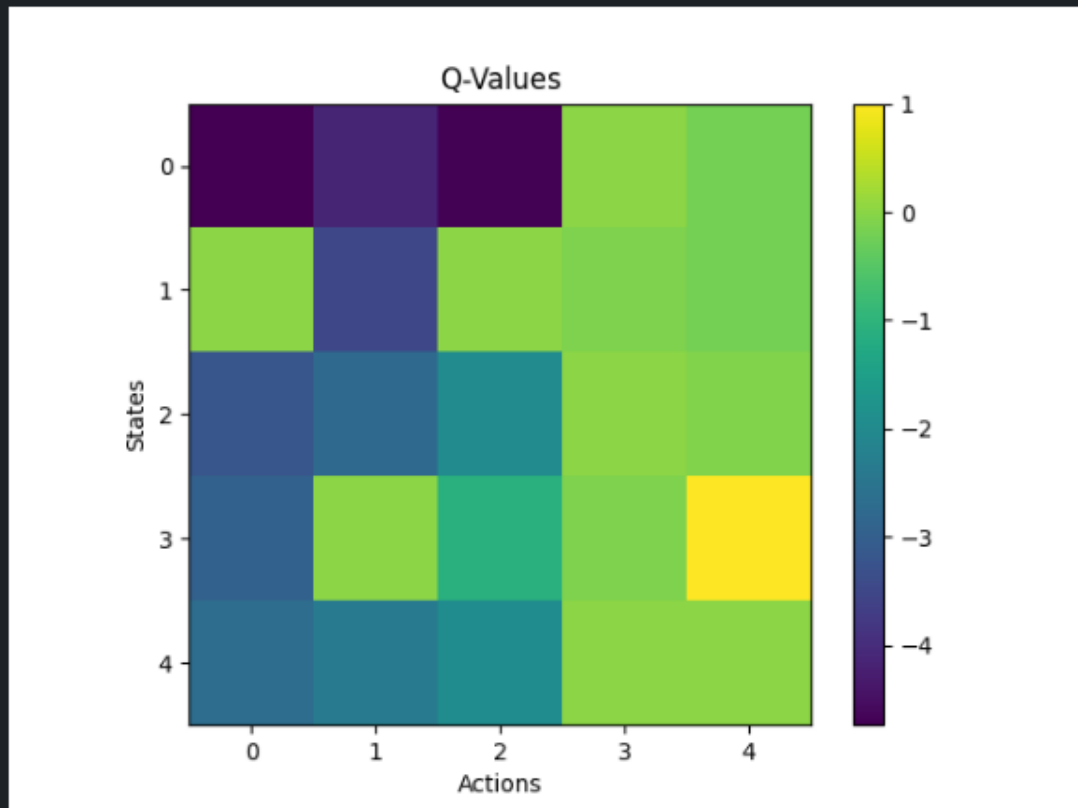
65
66 # Определяем лабиринт (0 - проходимая клетка, 1 - стена)
67 maze_array = np.array([[0, 0, 0, 1, 0],
68                        [1, 0, 1, 0, 0],
69                        [0, 0, 0, 1, 0],
70                        [0, 1, 0, 0, 0],
71                        [0, 0, 0, 1, 0]])
72
73 # Создаем лабиринт и обучаем Q-агента
74 maze = Maze(maze_array)
75 agent = QLearning(maze)
76
77 # Обучаем агента
78 agent.train(1000)
79
80 # Визуализация Q-таблицы
81 plt.imshow(np.max(agent.q_table, axis=2))
82 plt.colorbar()
83 plt.title("Q-Values")
84 plt.xlabel("Actions")
85 plt.ylabel("States")
86 plt.show()
87
88 # Тестирование агента
89 state = maze.reset()
90 path = [state]
91
92 while not maze.is_done():
93     action = np.argmax(agent.q_table[state])
94     next_state, _, _ = maze.step(action)
95     path.append(next_state)
96     state = next_state
97
98 print("Path to goal:", path)

```

Output

Path to goal: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (3, 2), (3, 3), (3, 4), (4, 4)]

[Execution complete with exit code 0]



Пояснения к коду:

1. **Maze Class:** Этот класс описывает лабиринт и его поведение. Он отвечает за инициализацию лабиринта, сброс состояния, выполнение действий и проверку, достигнута ли цель.
2. **QLearning Class:** Этот класс реализует алгоритм Q-обучения. Он содержит методы для выбора действия, обновления Q-таблицы и обучения агента.
3. **Обучение:** Агент обучается на 1000 эпизодов, после чего можно визуализировать Q-таблицу и протестировать агента.
4. **Визуализация:** Используется matplotlib для отображения значений Q.

Содержание работы.

1. Что называют марковским процессом принятия решений (МППР)?
2. На чем основано большинство алгоритмов МППР?
3. Какая стратегия называется оптимальной?
4. Реализуйте алгоритм прохождения лабиринта на Python.
5. Осуществите визуализацию отображения значений Q .
6. Проанализируйте результаты, полученные в п. 4 и 5.